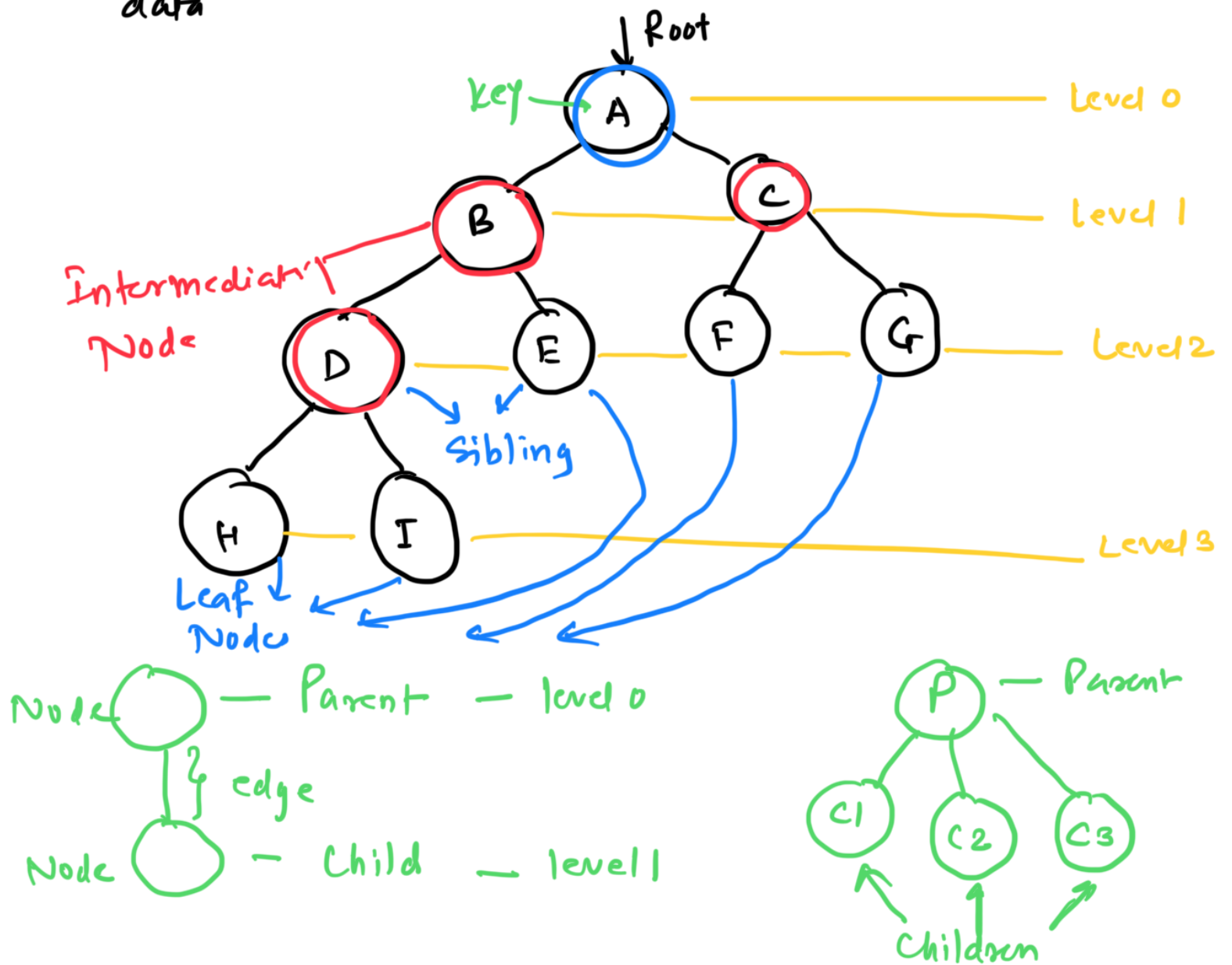


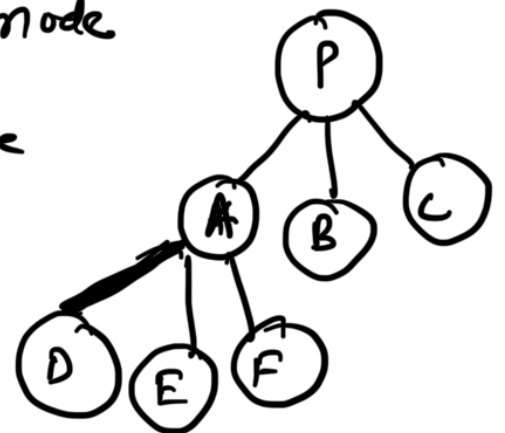
Trees:

- A hierarchical structure to represent and organize data



- Composed of nodes connected by edges
- Top most is called as root node
- Multiple children in a tree

Basic Terminologies



- Parent Node - Node has one / more children
- child Node - Direct descendent of Parent node
- Root Node - Topmost node with no parent
- Leaf Node - Node with no children
- Ancestor - predecessor in the path from the root node
- Descendent - Successor in the path from

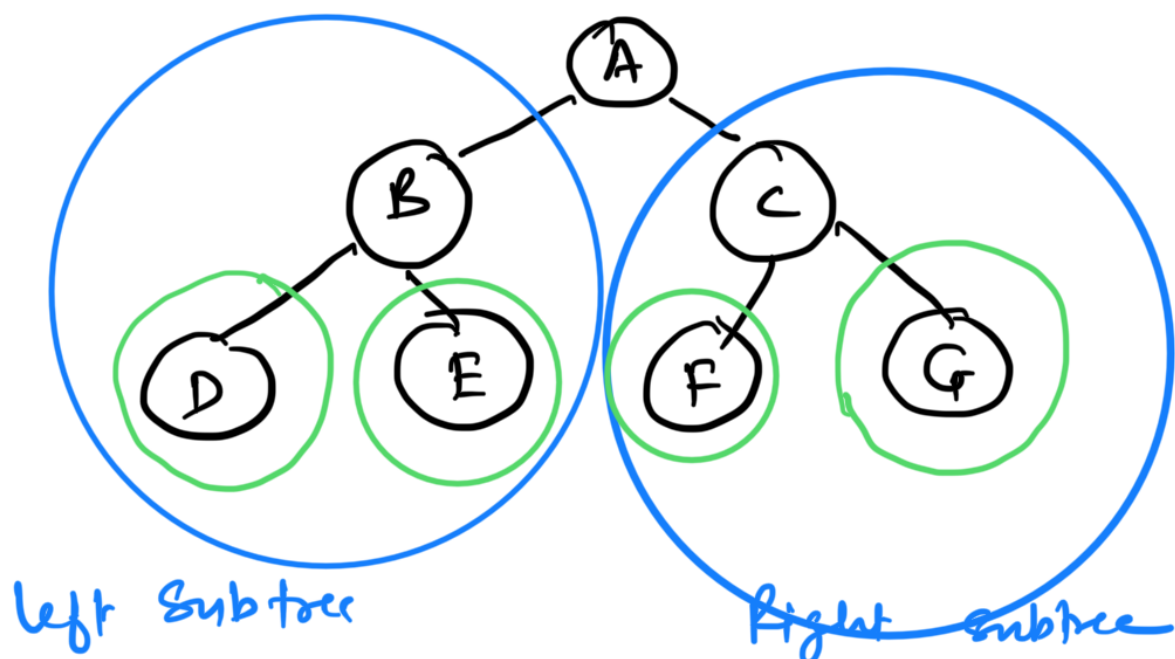
the root node

Sibling - Nodes that share the same parent

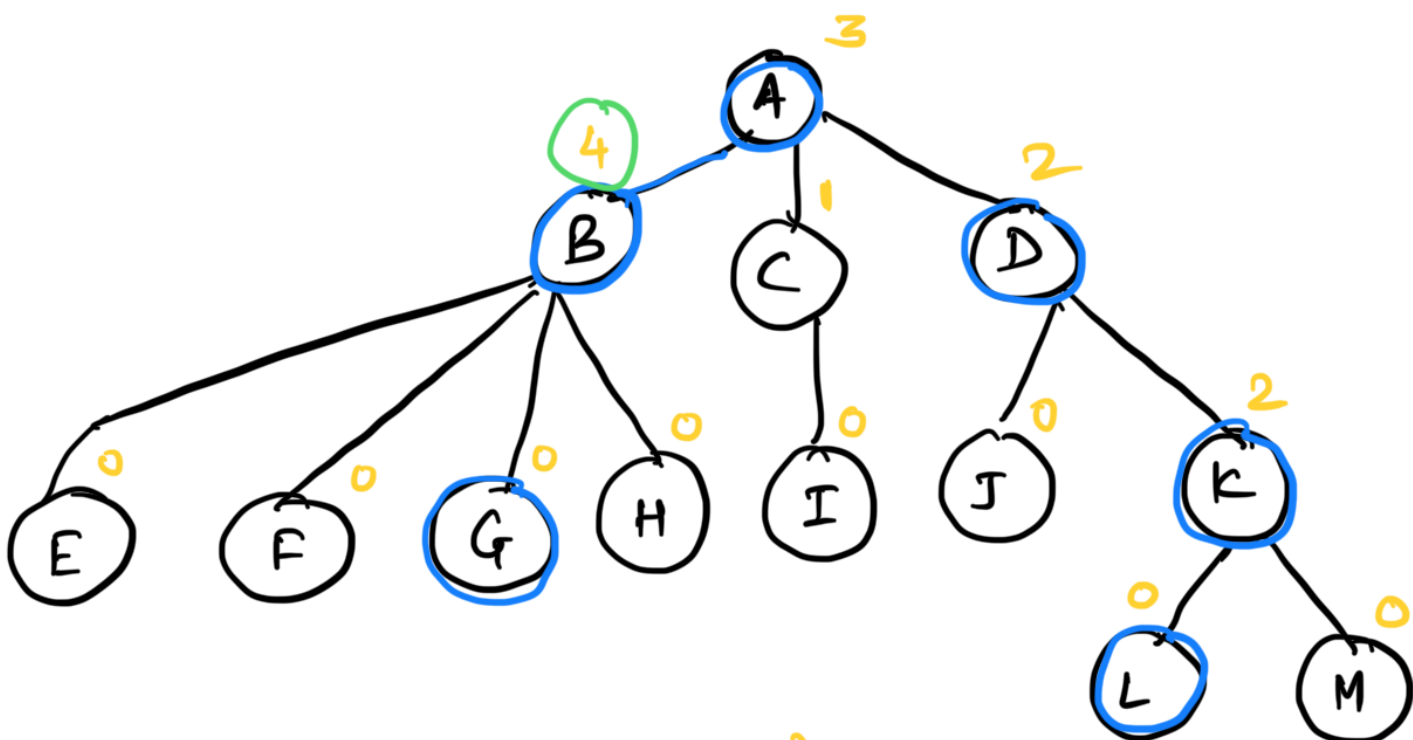
Level of a Node - The depth of the node measured by edges from the root.

Internal Node - Node with at least one child

Neighbor - Parent or child nodes of a tree



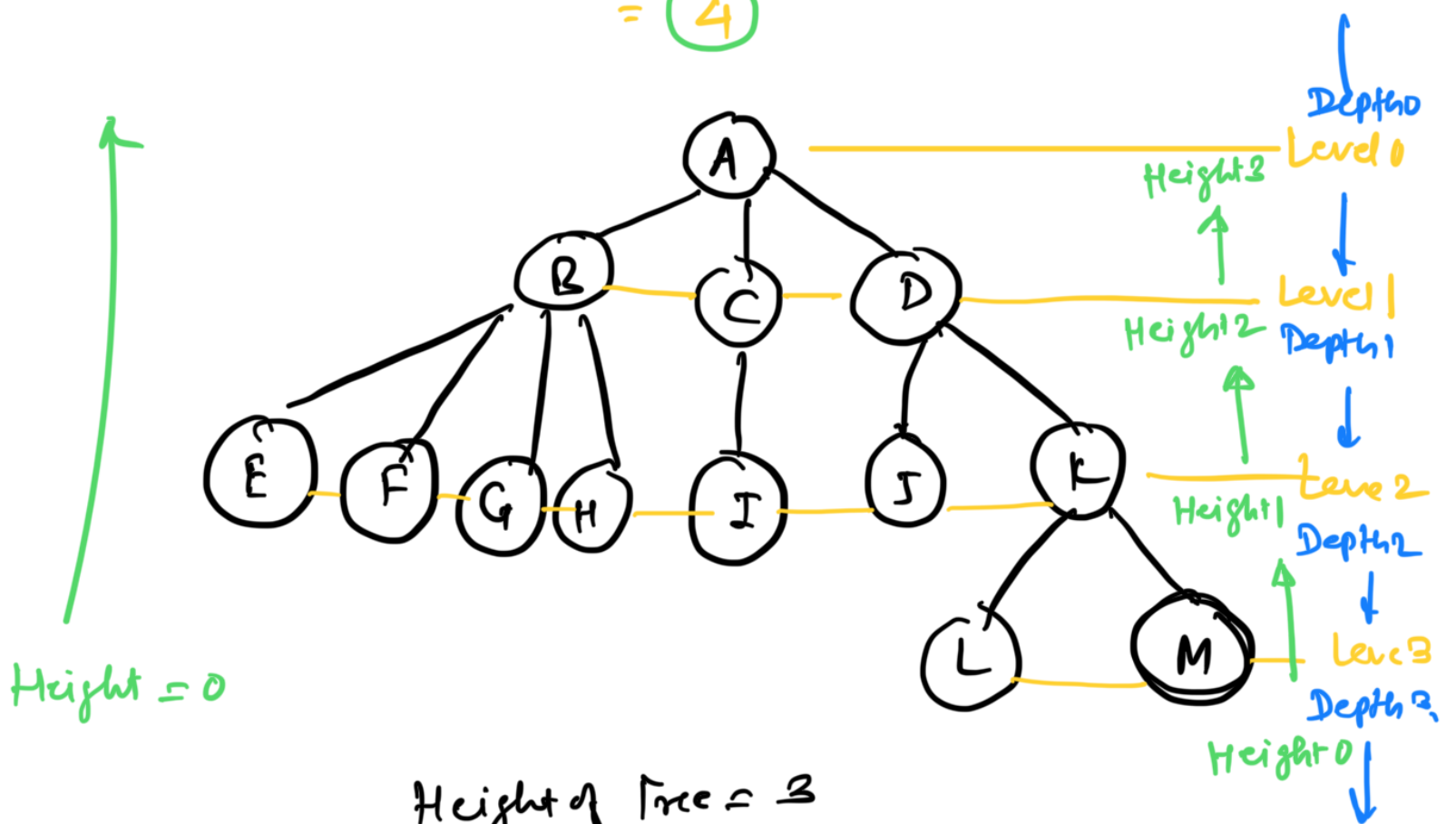
Subtree - A node along with its descendants



Degree of Node - $(A) = 3$

Degree of Tree - highest degree of node in tree

= (4)

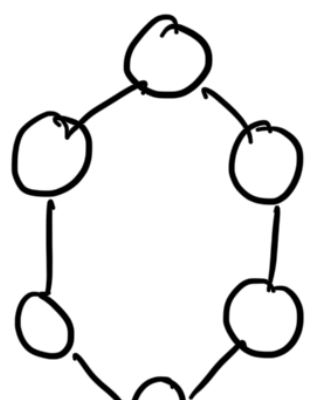
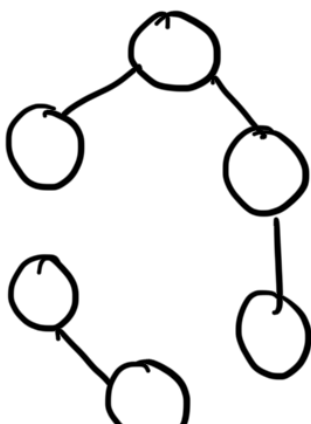
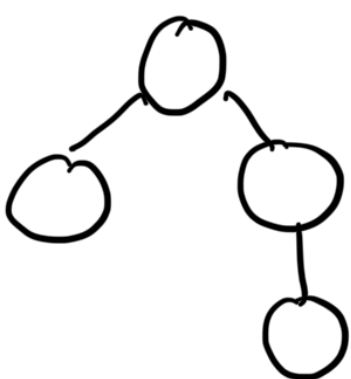


Level - Root = level 0 and node onwards will be on level 1 (incremental), so, on

Height - The number of nodes (excluding the node) on the longest path from the node to a leaf node

Properties of Tree

1. No. of nodes in a tree must be finite and nonempty set
2. There must be a path exist between every node of a tree
3. There must not be any cycle in the tree.



Tree

Not a tree



Not a tree



Importance - Tree

1. Naturally form hierarchies. (eg. file system)
2. Support dynamic node allocation

Types of Tree Data Structures

1. Binary Tree - Each node has at most 2 children
2. Ternary Tree - Each node has at most 3 children
3. N-ary Tree - Each node has multiple children

Operations

1. Create: Construct a tree
2. Insert: Add a new node
3. Search: Find specific node in a tree
4. Traversal: Visit each node of the trees.
5. Delete: Delete a node

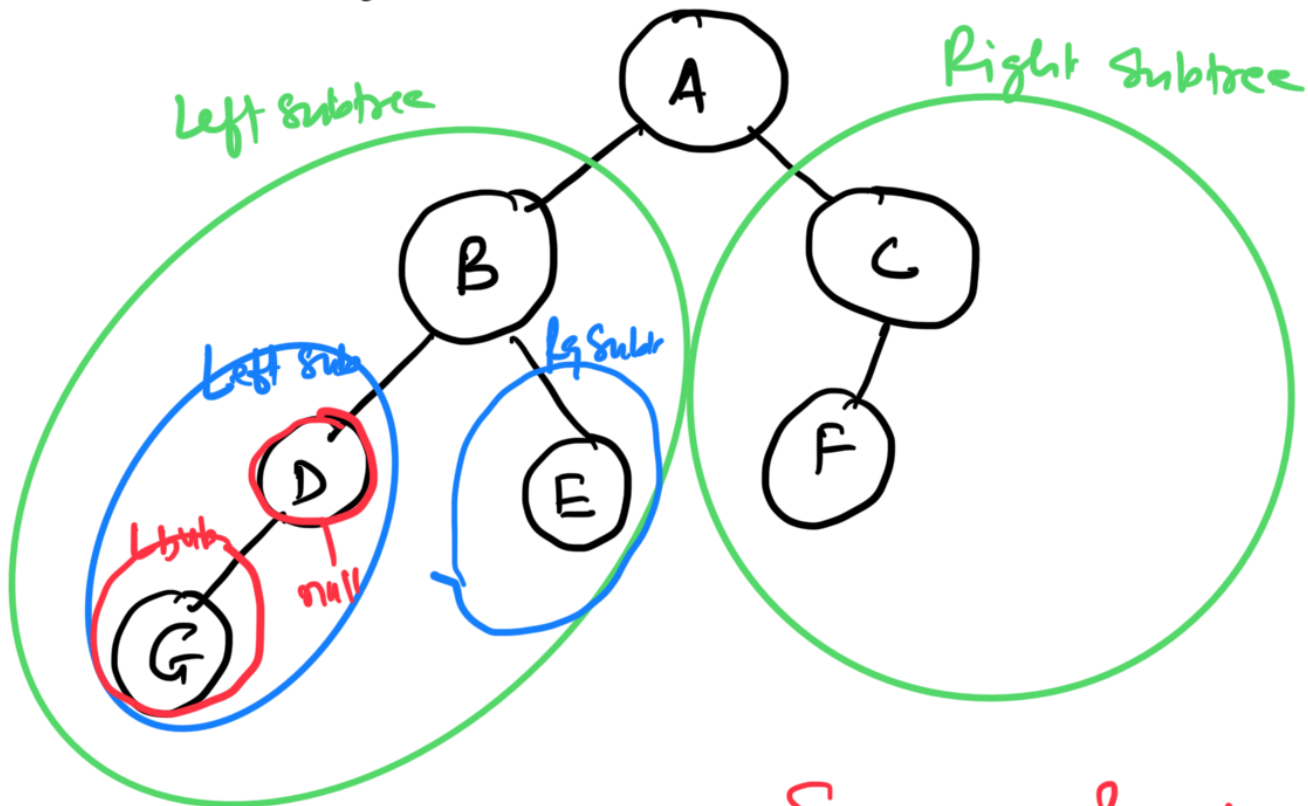
Tree Traversal Techniques

1. InOrder
2. PreOrder
3. PostOrder
4. DFS : Depth First Search

5. BFS: Breadth First Search

Binary Tree:

- hierarchical data structure, in which each node has at most two children, referred as left child and right child.

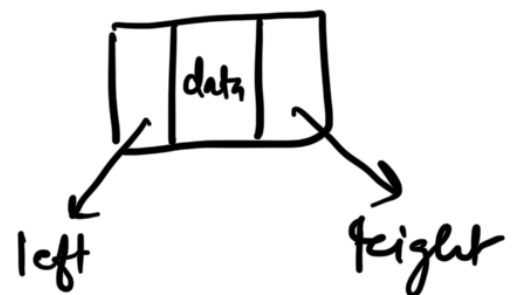


No of children = $\{0, 1, 2\}$ at most 2

Properties of Binary Tree:

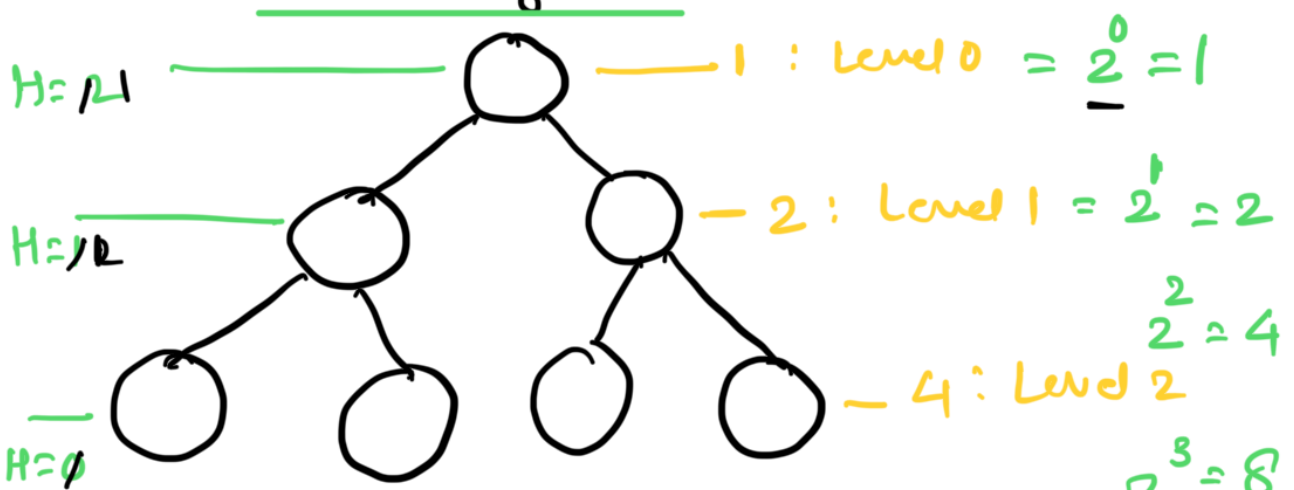
1. Basic Structure

- Data: key value
- Left child: Ref
- Right child: Ref



2. Maximum Nodes

- The max no. of nodes at level L is 2^L



- 8 : level 3

b. The max no of nodes in a binary tree of height h is $2^h - 1$ ($h=1$ / $h=0$)

$2^1 - 1 = 2 - 1 = 1 \Rightarrow$ Root Node $\uparrow$

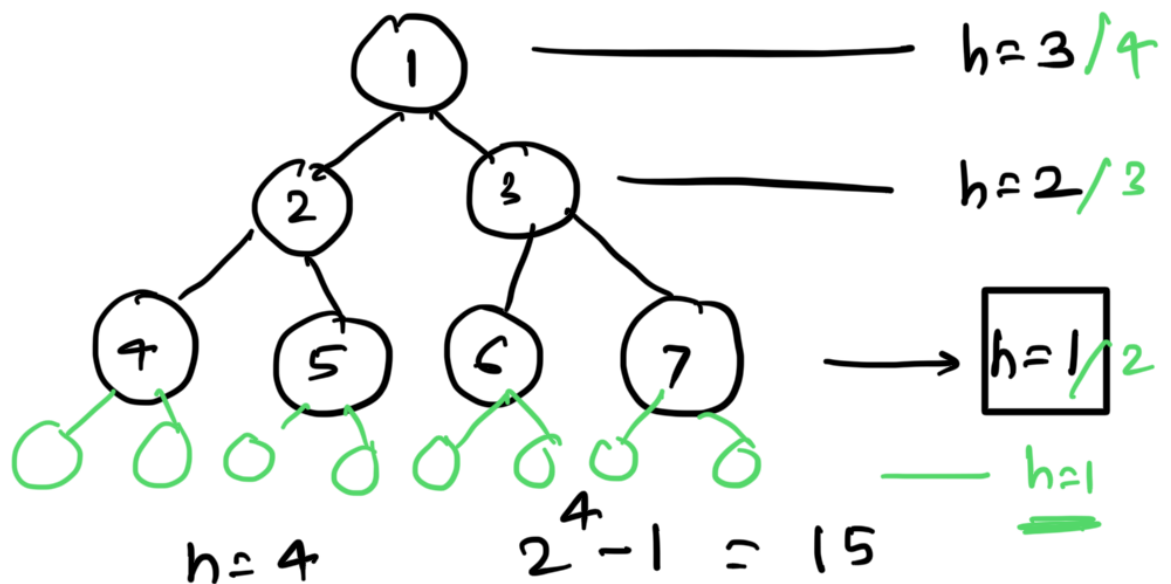
$h=0$	$2^h = 2^0 = 1$
$h=1$	$2^h = 2^1 = 2$
$h=2$	$2^h = 2^2 = 4$

$\Rightarrow$ depth / level.

Ex:

$h=3$

$2^3 - 1 = 7$



Note: $h=1$, when h is calculated for max no. of node with formula $2^h - 1$

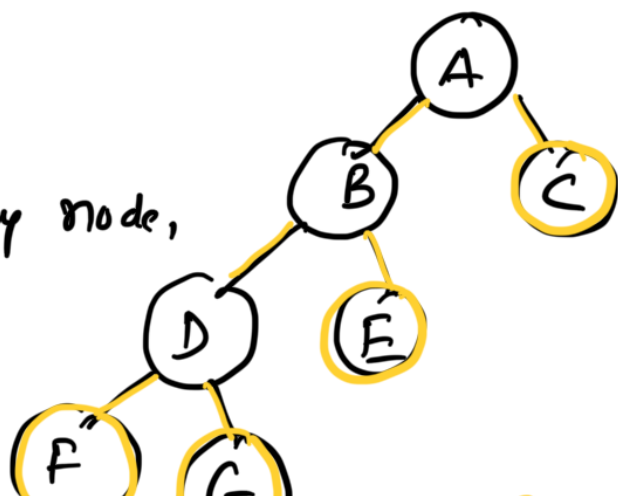
c. Maximum height of tree for N nodes:

$\log_2(N+1)$

Binary Tree:

1. Strict Binary Tree:

- A BT in which every node, except for the leaf nodes, has strictly



2 childrens.

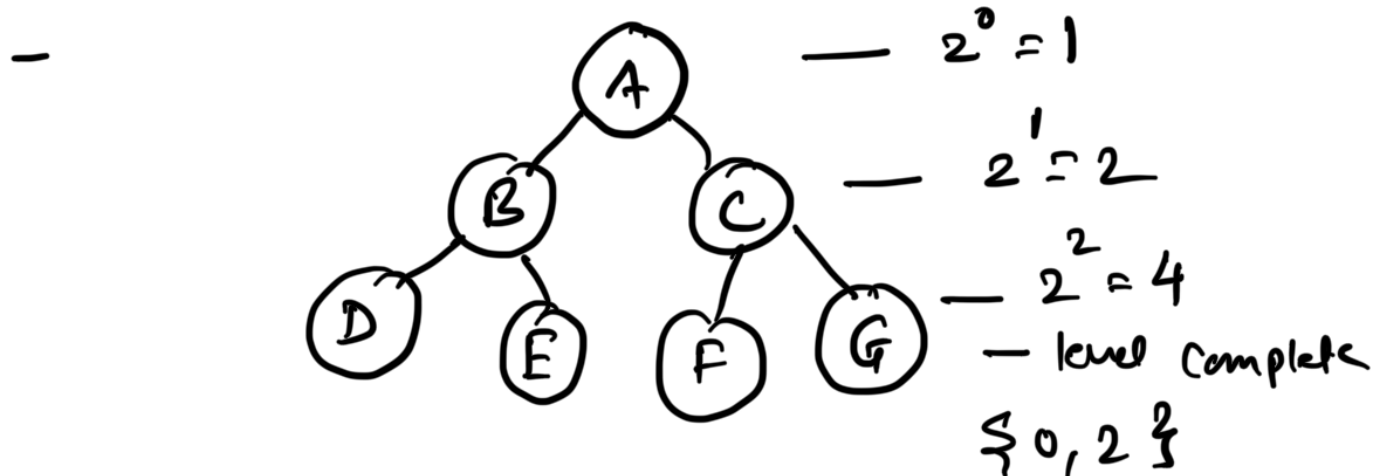
$\{0, 2\}$

strictly BT

— No node can have only one child

2. Full Binary Tree:

— A tree in which every node has either 0 or 2 children (similar to strict BT)

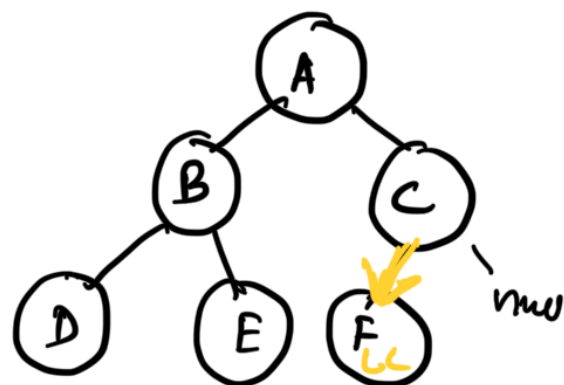


FBT $\rightarrow$ SBT $\rightarrow$ BT
 $\{0, 2\}$ $\{0, 2\}$
level complete

3. Complete Binary Tree:



Not a CBT: Incomplete BT



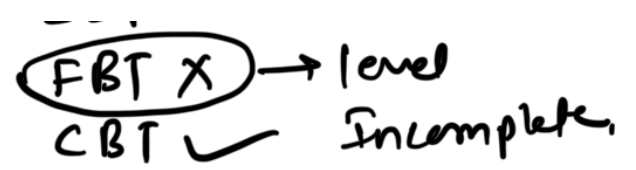
left child, right child
1st child
Complete BT

— CBT is a tree in which all levels, except possibly last, are completely filled, and all nodes are as far as left possible node in the last level.

4. Almost Complete Binary Tree (ACBT): Heap



BT ✓
SBT ✓



Perfect Binary Tree:

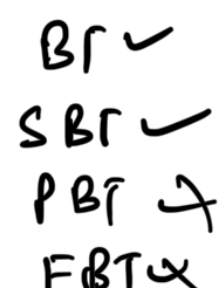
```

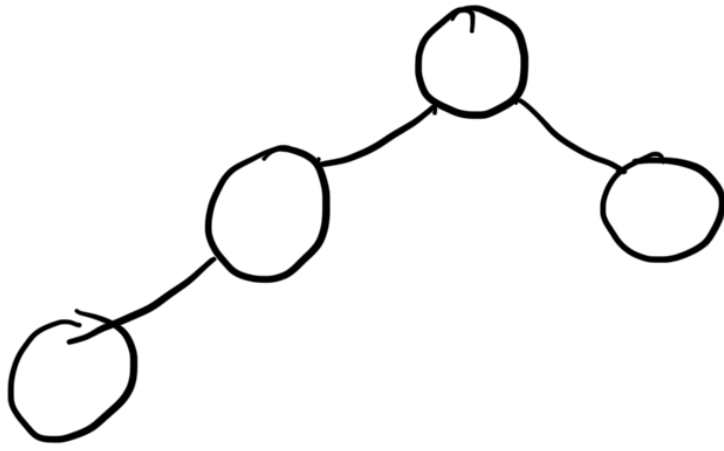
graph TD
    A((A)) --> B((B))
    A --> C((C))
    B --> D((D))
    B --> E((E))
    C --> F((F))
    C --> G((G))
  
```

[illegible]

BT ✓
SBT ✓
FBT ✗
PBT ✗
CBT ✓
ACBT ✓

BT	✓
SBT	✓
FRT	✗
PBT	✗
CBT	✓
AcBT	✓





CBT X
ACBT X

BT ✓
SBT X
FBT X
PBT X
CBT ✓
ACBT ✓